

Formato de Pruebas Estándar

Sistema de Gestión de Citas · Pruebas automatizadas con Jest

Este documento recoge la ejecución de las pruebas automatizadas del sistema, redactada con el formato estándar de casos de prueba. Cada ficha se cuenta en dos planos: los «Pasos de la prueba» describen el escenario tal como lo viviría el personal del laboratorio frente al sistema, y la fila «Cómo se automatiza con Jest» explica de qué manera la prueba reproduce ese mismo escenario sin que haya nadie delante de la pantalla. Todas las fichas corresponden a pruebas que existen en el repositorio; el video de evidencia muestra su ejecución en vivo.

1. Datos generales de la ejecución

Proyecto	Sistema de Gestión de Citas — El Arco Laboratorios
Módulo(s) evaluado(s)	Autenticación y Sesión · Pacientes · Citas y Recepción
Herramienta de prueba	Jest 29.7.0 (backend, frontend y servicio de WhatsApp)
Librerías de apoyo	Testing Library 16.3 y user-event 14.5 (interfaz) · Supertest 7.2 (rutas HTTP) · jest.mock para los dobles de prueba
Tipo de pruebas	Automatizadas: unitarias, de integración y de interfaz
Técnicas aplicadas	Caja blanca, caja negra y caja gris
Entorno de ejecución	MacBook Air · Apple M4 · 16 GB RAM · macOS 26.6.2 · Node.js v24.14.0
Navegador	Las pruebas de interfaz corren sobre jsdom, un navegador simulado con la misma API del DOM que Chrome; no requieren abrir un navegador real
Fecha de ejecución	11 de septiembre de 2026
Responsable	Alexander Pérez
Comando ejecutado	npm test — en backend, en frontend y en whatsapp-service
Resultado global	33 suites · 530 pruebas · 530 aprobadas · 0 reprobadas

2. Las tres técnicas empleadas y por qué

El sistema no se prueba con una sola técnica, porque no todos los riesgos se ven desde el mismo sitio. Cada ficha de este documento declara cuál se aplicó y lo justifica.

Caja negra	Se prueba el sistema por fuera, sin mirar el código: se entra por la interfaz o por la API, con los datos que usaría una persona, y se comprueba lo que devuelve. Aquí se usa sobre todo en las pantallas con Testing Library, que busca los campos por su etiqueta visible y el botón por su texto, igual que haría el personal del laboratorio. También en los casos de valores límite del teléfono y del DPI, que salen de la especificación del campo.
Caja blanca	Se prueba conociendo el código y recorriendo sus caminos a propósito. Es lo que permite afirmar cosas que por fuera no se ven: que la respuesta del login no lleva el hash de la contraseña, que una validación fallida no llegó a escribir en la base, o que un error a mitad de la recepción deshace lo ya creado. Son las pruebas unitarias del backend sobre los servicios.
Caja gris	Un punto intermedio: se comprueba un comportamiento visible desde fuera, pero el caso se diseñó sabiendo cómo está construido por dentro. Es el caso del mensaje de error del login, que debe ser idéntico para un usuario inexistente y para una contraseña equivocada precisamente porque por dentro son dos ramas distintas; o el de las fechas sugeridas cuando una cita se sale de la vigencia de la orden.

3. Módulo 1 — Autenticación y Sesión

Cubre el inicio de sesión, el trato que recibe la contraseña y las respuestas del sistema cuando las credenciales no son válidas. Se prueba por dentro (el servicio que verifica el hash y firma el token) y por fuera (la pantalla de login tal como la usa el personal).
Archivos de prueba: backend/src/tests/auth.service.test.js · frontend/src/tests/LoginPage.test.jsx

#AUT001	Autenticar un inicio de sesión exitoso en el sistema de gestión de citas.
ID del caso de prueba	#AUT001
Escenario de prueba	Autenticar un inicio de sesión exitoso en el sistema de gestión de citas.
Pasos de la prueba	1. El usuario abre el sistema en el navegador y llega a la pantalla de inicio de sesión. 2. El usuario introduce su nombre de usuario en el campo «Usuario». 3. El usuario introduce su contraseña en el campo «Contraseña». 4. El usuario hace clic en el botón «Ingresar». 5. El sistema verifica las credenciales y abre la sesión.
Requisitos previos	Una cuenta registrada y activa en el sistema, con su nombre de usuario y su contraseña.
Navegador / Dispositivo	MacBook Air · Apple M4 · 16 GB RAM · macOS 26.6.2 · Jest 29.7.0 · entorno «node» · Node.js v24.14.0 · sin navegador
Datos de prueba	Usuario «ana.lopez», contraseña «Laboratorio2026», rol Personal de citas, cuenta activa.
Resultados esperados	El sistema acepta las credenciales y abre la sesión: emite un pase de acceso (token) que lo identifica como el usuario 1, «ana.lopez», con rol de Personal de citas, y deja registrada la fecha de este último acceso.
Resultados reales	Como se esperaba. Las dos pruebas del caso se ejecutan en verde.
Estado de la prueba	APROBADO
Tipo de prueba	Caja blanca — unitaria (positiva) Se conoce y se recorre el interior del servicio: la consulta al modelo, la verificación del hash, la firma del token y el registro del acceso. La prueba abre el token y mira lo que lleva dentro, algo que el usuario nunca ve.
Cómo se automatiza con Jest	Jest sustituye el modelo de usuario por un doble de prueba que devuelve a «Ana López», activa y con su contraseña ya cifrada con bcrypt; así no hace falta base de datos. Se llama directamente a auth.service.login con el usuario y la contraseña, se descifra el token devuelto con jwt.verificar y se comprueba su contenido, además de verificar que se llamó a registrarAcceso.
Pruebas de Jest que lo cubren	· «devuelve un token válido con credenciales correctas» · «registra el último acceso»
Caso del plan de pruebas	CP-LOGIN-001 y CP-LOGIN-006
Cómo ejecutarlo solo	./caso.sh AUT001 Equivalente a: cd backend && npx jest auth.service -t "devuelve un token válido con credenciales correctas registra el último acceso" --verbose

#AUT002 Comprobar que al iniciar sesión el sistema no envía al navegador la contraseña cifrada del usuario.	
ID del caso de prueba	#AUT002
Escenario de prueba	Comprobar que al iniciar sesión el sistema no envía al navegador la contraseña cifrada del usuario.
Pasos de la prueba	1. El usuario introduce su nombre de usuario y su contraseña. 2. El usuario hace clic en «Ingresar». 3. El sistema responde con la sesión abierta y los datos del perfil del usuario. 4. Se revisa todo lo que contiene esa respuesta, tal como viajaría por la red hasta el navegador.
Requisitos previos	Una cuenta registrada y activa, cuya contraseña está guardada cifrada en la base de datos.
Navegador / Dispositivo	MacBook Air · Apple M4 · 16 GB RAM · macOS 26.6.2 · Jest 29.7.0 · entorno «node» · Node.js v24.14.0 · sin navegador
Datos de prueba	El mismo usuario «ana.lopez», cuyo registro en la base sí contiene la contraseña cifrada.
Resultados esperados	La respuesta trae el nombre, el usuario y el rol, pero no incluye en ningún sitio la contraseña cifrada, aunque el registro leído de la base de datos sí la tenga.
Resultados reales	Como se esperaba. El dato no aparece en la respuesta.
Estado de la prueba	APROBADO
Tipo de prueba	Caja blanca — unitaria (seguridad) La prueba sabe que el modelo devuelve la contraseña cifrada y comprueba que el servicio la retira antes de responder: es una afirmación sobre la implementación, no sobre lo que se ve en pantalla.
Cómo se automatiza con Jest	Con el mismo doble de prueba del caso anterior, se llama a <code>auth.service.login</code> y se comprueba que el objeto del usuario devuelto no tiene la propiedad <code>password_hash</code> .
Pruebas de Jest que lo cubren	· «nunca devuelve el hash de la contraseña»
Caso del plan de pruebas	CP-LOGIN-006
Cómo ejecutarlo solo	./caso.sh AUT002 Equivale a: <code>cd backend && npx jest auth.service -t "nunca devuelve el hash de la contraseña" --verbose</code>

#AUT003 Rechazar el acceso de quien escribe mal su contraseña y el de quien tiene la cuenta desactivada.	
ID del caso de prueba	#AUT003
Escenario de prueba	Rechazar el acceso de quien escribe mal su contraseña y el de quien tiene la cuenta desactivada.
Pasos de la prueba	1. El usuario abre la pantalla de inicio de sesión. 2. El usuario introduce su nombre de usuario correcto y una contraseña equivocada, y hace clic en «Ingresar». 3. Después, un usuario al que el administrador desactivó la cuenta introduce sus credenciales correctas. 4. Ese segundo usuario hace clic en «Ingresar».

Requisitos previos	Una cuenta activa y otra cuenta desactivada por el administrador, ambas con su contraseña conocida.
Navegador / Dispositivo	MacBook Air · Apple M4 · 16 GB RAM · macOS 26.6.2 · Jest 29.7.0 · entorno «node» · Node.js v24.14.0 · sin navegador
Datos de prueba	Intento 1: usuario correcto con la contraseña «incorrecta». Intento 2: usuario y contraseña correctos, pero la cuenta está desactivada.
Resultados esperados	Ninguno de los dos entra. El sistema responde «no autorizado» en ambos casos y no abre sesión. Una cuenta desactivada no puede entrar por mucho que acierte la contraseña.
Resultados reales	Como se esperaba. Los dos intentos son rechazados con el código 401.
Estado de la prueba	APROBADO
Tipo de prueba	Caja blanca — unitaria (negativa) Se recorren a propósito las dos ramas de rechazo que tiene escritas el servicio, elegidas conociendo el código.
Cómo se automatiza con Jest	Se llama a auth.service.login con la contraseña equivocada y se comprueba que la llamada termina en un error con statusCode 401. El segundo intento usa el mismo doble de prueba pero con el campo activo en 0.
Pruebas de Jest que lo cubren	· «rechaza una contraseña incorrecta» · «rechaza a un usuario inactivo aunque la contraseña sea correcta»
Caso del plan de pruebas	CP-LOGIN-002
Cómo ejecutarlo solo	./caso.sh AUT003 Equivale a: cd backend && npx jest auth.service -t "rechaza una contraseña incorrecta rechaza a un usuario inactivo" --verbose

#AUT004 Comprobar que el mensaje de error del inicio de sesión no permite averiguar qué usuarios existen en el sistema.	
ID del caso de prueba	#AUT004
Escenario de prueba	Comprobar que el mensaje de error del inicio de sesión no permite averiguar qué usuarios existen en el sistema.
Pasos de la prueba	1. Alguien ajeno al laboratorio abre la pantalla de inicio de sesión. 2. Introduce un nombre de usuario inventado, que no existe, y cualquier contraseña. Hace clic en «Ingresar». 3. Lee el mensaje de error que aparece. 4. Introduce ahora un nombre de usuario que sí existe, pero con una contraseña equivocada. Hace clic en «Ingresar». 5. Compara los dos mensajes de error.
Requisitos previos	Una cuenta registrada en el sistema y un nombre de usuario que no corresponde a nadie.
Navegador / Dispositivo	MacBook Air · Apple M4 · 16 GB RAM · macOS 26.6.2 · Jest 29.7.0 · entorno «node» · Node.js v24.14.0 · sin navegador
Datos de prueba	Intento 1: usuario «nadie», que no existe. Intento 2: usuario «ana.lopez», que sí existe, con la contraseña «mala».
Resultados esperados	Los dos intentos devuelven exactamente el mismo mensaje, de modo que quien lo lea no pueda distinguir «ese usuario no existe» de «esa contraseña está mal». Tampoco queda registrado ningún acceso en ninguno de los dos casos.

Resultados reales	Como se esperaba. Los mensajes coinciden y no se registra acceso.
Estado de la prueba	APROBADO
Tipo de prueba	Caja gris — seguridad (enumeración de usuarios) Es gris porque se comprueba algo visible desde fuera —el mensaje que recibiría un atacante— pero el caso se diseñó conociendo por dentro que existen dos ramas distintas que debían converger en la misma respuesta.
Cómo se automatiza con Jest	Se hace que la búsqueda de usuario devuelva un resultado vacío en el primer intento y el usuario real en el segundo. Se capturan los dos errores y se comparan sus mensajes; además se verifica que registrarAcceso no llegó a llamarse.
Pruebas de Jest que lo cubren	· «usa el mismo mensaje para usuario inexistente y contraseña incorrecta» · «no registra acceso si las credenciales fallan»
Caso del plan de pruebas	CP-LOGIN-003
Cómo ejecutarlo solo	./caso.sh AUT004 Equivale a: cd backend && npx jest auth.service -t "usa el mismo mensaje para usuario inexistente no registra acceso si las credenciales fallan" --verbose

#AUT005 Autenticar un inicio de sesión exitoso desde la pantalla de login y llegar al Panel del día.	
ID del caso de prueba	#AUT005
Escenario de prueba	Autenticar un inicio de sesión exitoso desde la pantalla de login y llegar al Panel del día.
Pasos de la prueba	1. El personal de citas abre el sistema y llega a la pantalla de inicio de sesión. 2. El usuario introduce «ana.lopez» en el campo «Usuario». 3. El usuario introduce su contraseña en el campo «Contraseña». 4. El usuario hace clic en el botón «Ingresar». 5. El sistema abre la sesión y cambia de pantalla.
Requisitos previos	Una cuenta registrada y activa en el sistema.
Navegador / Dispositivo	MacBook Air · Apple M4 · 16 GB RAM · macOS 26.6.2 · Jest 29.7.0 · entorno «jsdom» (navegador simulado) · Testing Library 16.3 + user-event 14.5 · Node.js v24.14.0
Datos de prueba	Usuario «ana.lopez» y contraseña «Laboratorio2026», ambos válidos.
Resultados esperados	El formulario envía al sistema exactamente el usuario y la contraseña que se escribieron, y una vez dentro la aplicación muestra el Panel del día, que es el resumen con el que empieza el turno. Ocurre así incluso si el usuario había llegado al login desde otra pantalla.
Resultados reales	Como se esperaba. Se envían esos datos y se aterriza en el Panel.
Estado de la prueba	APROBADO
Tipo de prueba	Caja negra — funcional (ruta ideal) Se interactúa con la pantalla igual que una persona —buscando los campos por su etiqueta visible y el botón por su texto— sin mirar cómo está escrito el componente por dentro.
Cómo se automatiza con Jest	La pantalla se dibuja en un navegador simulado (jsdom) dentro de su proveedor de sesión y su enrutador. user-event escribe en los campos y pulsa el botón como lo haría una persona. El servicio de autenticación está sustituido por un doble que responde con éxito, así que no hace falta levantar la API ni la base de datos; después se comprueba qué recibió ese servicio y qué pantalla quedó visible.
Pruebas de Jest que lo cubren	· «envía las credenciales al servicio de autenticación»

	· «lleva al panel después de entrar» · «lleva al panel aunque se llegara al login desde otra pantalla»
Caso del plan de pruebas	CP-LOGIN-001 y CP-LOGIN-008
Cómo ejecutarlo solo	./caso.sh AUT005 Equivale a: cd frontend && BABEL_ENV=test npx jest LoginPage -t "envía las credenciales al servicio lleva al panel" --verbose

#AUT006 La contraseña se escribe oculta y el botón «Ver la contraseña» permite revisarla sin enviar el formulario.

ID del caso de prueba	#AUT006
Escenario de prueba	La contraseña se escribe oculta y el botón «Ver la contraseña» permite revisarla sin enviar el formulario.
Pasos de la prueba	1. El usuario abre la pantalla de inicio de sesión. 2. El usuario escribe su contraseña y observa que aparece oculta, en puntos. 3. El usuario hace clic en el botón «Ver la contraseña» para revisar si la escribió bien. 4. El usuario hace clic en «Ocultar la contraseña» para volver a taparla. 5. En una pantalla recién abierta, el usuario pulsa solo «Ver la contraseña», sin haber llenado nada.
Requisitos previos	Ninguno: la pantalla de inicio de sesión es pública.
Navegador / Dispositivo	MacBook Air · Apple M4 · 16 GB RAM · macOS 26.6.2 · Jest 29.7.0 · entorno «jsdom» (navegador simulado) · Testing Library 16.3 + user-event 14.5 · Node.js v24.14.0
Datos de prueba	Contraseña de prueba «Laboratorio2026».
Resultados esperados	La contraseña se escribe oculta. Al pulsar «Ver la contraseña» se muestra en claro, conservando lo ya escrito, y al pulsar «Ocultar la contraseña» vuelve a taparse. Pulsar ese botón nunca intenta iniciar sesión: no envía el formulario a medio llenar.
Resultados reales	Como se esperaba en los tres momentos.
Estado de la prueba	APROBADO
Tipo de prueba	Caja negra — funcional y de usabilidad Todo lo que se afirma es observable en la interfaz: lo que ve el usuario y lo que ocurre al pulsar.
Cómo se automatiza con Jest	En el navegador simulado se comprueba el tipo del campo antes y después de pulsar cada botón, y que el valor escrito se conserva. En la última variante se verifica que el servicio de autenticación no llegó a llamarse.
Pruebas de Jest que lo cubren	· «oculta la contraseña mientras se escribe» · «la muestra y la vuelve a ocultar» · «no envía el formulario»
Caso del plan de pruebas	CP-LOGIN-004
Cómo ejecutarlo solo	./caso.sh AUT006 Equivale a: cd frontend && BABEL_ENV=test npx jest LoginPage -t "oculta la contraseña mientras se escribe la muestra y la vuelve a ocultar no envía el formulario" --verbose

#AUT007 Un usuario cambia su propia contraseña, y el sistema se lo impide si no acierta la actual.	
ID del caso de prueba	#AUT007
Escenario de prueba	Un usuario cambia su propia contraseña, y el sistema se lo impide si no acierta la actual.
Pasos de la prueba	1. El usuario, ya con la sesión iniciada, abre la opción de cambiar su contraseña. 2. El usuario escribe su contraseña actual. 3. El usuario escribe la contraseña nueva que quiere usar. 4. El usuario guarda el cambio. 5. Después, el usuario repite la operación pero escribiendo mal su contraseña actual.
Requisitos previos	Sesión iniciada con una cuenta cuya contraseña actual se conoce.
Navegador / Dispositivo	MacBook Air · Apple M4 · 16 GB RAM · macOS 26.6.2 · Jest 29.7.0 · entorno «node» · Node.js v24.14.0 · sin navegador
Datos de prueba	Contraseña actual correcta: «Laboratorio2026». Nueva: «NuevaClave2026». Contraseña actual equivocada: «equivocada».
Resultados esperados	Con la contraseña actual correcta el cambio se guarda, y lo que queda almacenado es la nueva contraseña cifrada, nunca su texto tal cual. Con la contraseña actual equivocada el sistema rechaza el cambio y no modifica nada.
Resultados reales	Como se esperaba en los dos escenarios.
Estado de la prueba	APROBADO
Tipo de prueba	Caja blanca — unitaria (positiva y negativa) La prueba inspecciona el argumento exacto con el que se llamó al modelo y verifica el cifrado guardado: eso solo puede hacerse conociendo la implementación.
Cómo se automatiza con Jest	Se llama a <code>auth.service.cambiarContrasenaPropia</code> y se recoge la llamada que recibió el modelo. Se comprueba que el valor guardado no es el texto plano y que sí se corresponde con la contraseña nueva al verificarlo con <code>bcrypt</code> . En el caso negativo se comprueba que actualizar no llegó a llamarse.
Pruebas de Jest que lo cubren	· «<i>actualiza el hash cuando la contraseña actual es correcta</i>» · «<i>rechaza el cambio si la contraseña actual no coincide</i>»
Caso del plan de pruebas	CP-USU-008
Cómo ejecutarlo solo	<code>./caso.sh AUT007</code> Equivale a: <code>cd backend && npx jest auth.service -t "cambiarContrasenaPropia" --verbose</code>

4. Módulo 2 — Pacientes

Cubre el alta y la edición de pacientes: la normalización del teléfono y del DPI, los valores límite de ambos campos, los avisos por duplicado y la validación en el formulario antes de molestar al servidor.

Archivos de prueba: backend/src/tests/paciente.service.test.js · frontend/src/tests/PacienteFormDialog.test.jsx

#PAC001 Registrar un paciente nuevo escribiendo el teléfono con guion y el nombre con espacios de más.	
ID del caso de prueba	#PAC001
Escenario de prueba	Registrar un paciente nuevo escribiendo el teléfono con guion y el nombre con espacios de más.
Pasos de la prueba	1. El personal de citas entra al módulo «Pacientes». 2. Hace clic en «Nuevo paciente». 3. Escribe el nombre del paciente dejando espacios de más al principio y al final, como ocurre al copiar y pegar. 4. Escribe el teléfono con guion, tal como la gente lo dicta: «5555-5555». 5. Hace clic en «Guardar».
Requisitos previos	Sesión iniciada con permiso para registrar pacientes.
Navegador / Dispositivo	MacBook Air · Apple M4 · 16 GB RAM · macOS 26.6.2 · Jest 29.7.0 · entorno «node» · Node.js v24.14.0 · sin navegador
Datos de prueba	Nombre « Ana López » y teléfono «5555-5555».
Resultados esperados	El paciente queda registrado con el teléfono limpio, «55555555», y con el nombre «Ana López» sin los espacios sobrantes. El sistema no obliga al personal a escribir en un formato concreto: lo ordena él.
Resultados reales	Como se esperaba en ambos casos.
Estado de la prueba	APROBADO
Tipo de prueba	Caja blanca — unitaria (positiva) Se comprueba la transformación interna que aplica el servicio antes de escribir en la base, no lo que se ve en pantalla.
Cómo se automatiza con Jest	El modelo de paciente está sustituido por un doble que devuelve lo que se le pidió guardar, así que cada caso queda independiente y no hace falta base de datos. Se llama a paciente.service.crear y se inspeccionan el teléfono y el nombre del paciente resultante.
Pruebas de Jest que lo cubren	· «guarda el teléfono normalizado» · «recorta los espacios del nombre»
Caso del plan de pruebas	CP-PAC-001
Cómo ejecutarlo solo	./caso.sh PAC001 Equivale a: cd backend && npx jest paciente.service -t "guarda el teléfono normalizado recorta los espacios del nombre" --verbose

#PAC002 Intentar registrar pacientes con teléfonos que no son válidos en Guatemala: uno corto, uno largo, uno con letras y uno vacío.

ID del caso de prueba	#PAC002
Escenario de prueba	Intentar registrar pacientes con teléfonos que no son válidos en Guatemala: uno corto, uno largo, uno con letras y uno vacío.
Pasos de la prueba	1. El personal de citas abre «Nuevo paciente». 2. Escribe el nombre del paciente. 3. Escribe un teléfono de 7 dígitos, uno menos del mínimo, y hace clic en «Guardar». 4. Repite la operación con un teléfono de 9 dígitos, uno más del máximo. 5. Repite escribiendo letras en lugar de números. 6. Repite dejando el campo del teléfono vacío.
Requisitos previos	Sesión iniciada con permiso para registrar pacientes. En Guatemala el teléfono válido tiene exactamente 8 dígitos.
Navegador / Dispositivo	MacBook Air · Apple M4 · 16 GB RAM · macOS 26.6.2 · Jest 29.7.0 · entorno «node» · Node.js v24.14.0 · sin navegador
Datos de prueba	Los cuatro teléfonos inválidos: «2323232», «232323233», «abcdefgh» y el campo vacío.
Resultados esperados	Los cuatro intentos son rechazados con un aviso de dato inválido y ninguno llega a registrar al paciente. La comprobación ocurre antes de tocar la base de datos, así que no queda ninguna ficha a medio crear.
Resultados reales	Como se esperaba. Las cuatro variantes son rechazadas.
Estado de la prueba	APROBADO
Tipo de prueba	Caja negra — partición de equivalencia y valores límite Los casos salen de la especificación del campo —un número de Guatemala de 8 dígitos— y se eligen los bordes de esa regla, sin mirar cómo está escrita la validación.
Cómo se automatiza con Jest	Una sola prueba escrita con <code>it.each</code> recorre los cuatro valores. Para cada uno se comprueba que la llamada termina en un error 400 y que el modelo nunca recibió la orden de crear.
Pruebas de Jest que lo cubren	· «rechaza el teléfono inválido» (cuatro variantes con <code>it.each</code>)
Caso del plan de pruebas	CP-PAC-002
Cómo ejecutarlo solo	<code>./caso.sh PAC002</code> Equivale a: <code>cd backend && npx jest paciente.service -t "rechaza el teléfono inválido" --verbose</code>

#PAC003 Registrar a un familiar que comparte el número de teléfono con un paciente ya registrado.

ID del caso de prueba	#PAC003
Escenario de prueba	Registrar a un familiar que comparte el número de teléfono con un paciente ya registrado.
Pasos de la prueba	1. Llega al mostrador la hija de una paciente que ya está registrada, y da el mismo número de teléfono de la casa. 2. El personal de citas abre «Nuevo paciente». 3. Escribe el nombre de la hija. 4. Escribe el teléfono, que ya pertenece a otro paciente. 5. Hace clic en «Guardar» y lee lo que responde el sistema.
Requisitos previos	Existe ya un paciente registrado con ese mismo número de teléfono.
Navegador / Dispositivo	MacBook Air · Apple M4 · 16 GB RAM · macOS 26.6.2 · Jest 29.7.0 · entorno «node» · Node.js v24.14.0 · sin navegador

Datos de prueba	Paciente nuevo «Ana Pérez» con el teléfono «55555555», ya usado por «Carlos Pérez».
Resultados esperados	El paciente se registra igualmente, y el sistema muestra un aviso que nombra al paciente que ya usaba ese número. En el laboratorio los familiares comparten teléfono: es una advertencia para el mostrador, no un error que corte el registro.
Resultados reales	Como se esperaba. Se crea el paciente y el aviso nombra a Carlos Pérez.
Estado de la prueba	APROBADO
Tipo de prueba	Caja gris — regla de negocio La regla se comprueba sobre la respuesta del servicio, pero el caso se diseñó sabiendo que existe una consulta previa por teléfono cuyo resultado no debe bloquear la escritura.
Cómo se automatiza con Jest	Se prepara la búsqueda por teléfono para que devuelva un paciente existente. Después se crea al paciente nuevo y se comprueba que la respuesta trae tanto el paciente creado como una lista de avisos que menciona al anterior.
Pruebas de Jest que lo cubren	· «avisa si otro paciente ya usa ese teléfono, pero no bloquea el registro»
Caso del plan de pruebas	CP-PAC-003
Cómo ejecutarlo solo	./caso.sh PAC003 Equivale a: cd backend && npx jest paciente.service -t "avisa si otro paciente ya usa ese teléfono" --verbose

#PAC004 Registrar el DPI del paciente: copiado con espacios, omitido por ser opcional, y con longitudes equivocadas.

ID del caso de prueba	#PAC004
Escenario de prueba	Registrar el DPI del paciente: copiado con espacios, omitido por ser opcional, y con longitudes equivocadas.
Pasos de la prueba	1. El personal de citas abre «Nuevo paciente». 2. Copia el DPI del documento del paciente tal cual, con los espacios que trae: «1111 11111 1111». Guarda. 3. Registra a otro paciente sin escribir ningún DPI, porque no lo lleva encima. 4. Registra a otro escribiendo un DPI de 10 dígitos. 5. Repite con un DPI de 12 dígitos y luego con uno de 14.
Requisitos previos	Sesión iniciada. El DPI de Guatemala tiene exactamente 13 dígitos y en este sistema es un dato opcional.
Navegador / Dispositivo	MacBook Air · Apple M4 · 16 GB RAM · macOS 26.6.2 · Jest 29.7.0 · entorno «node» · Node.js v24.14.0 · sin navegador
Datos de prueba	Válido: «1111 11111 1111». Sin DPI: el campo vacío. Inválidos: «1111111111» (10), «11111111111» (12) y «11111111111111» (14).
Resultados esperados	El DPI válido se guarda limpio, como «111111111111», sin los espacios. El paciente sin DPI se registra igualmente, porque el campo es opcional. Las tres longitudes incorrectas son rechazadas con un aviso y no registran nada.
Resultados reales	Como se esperaba en las siete variantes.
Estado de la prueba	APROBADO
Tipo de prueba	Caja negra — valores límite (con verificación de caja blanca)

	Las longitudes 10, 12 y 14 rodean el valor válido de 13, que es el criterio del documento de identificación; se añade la comprobación interna de que la escritura no llega a ocurrir.
Cómo se automatiza con Jest	Las variantes se recorren con <code>it.each</code> . El caso «sin DPI» se prueba en sus tres formas técnicas: campo ausente, cadena vacía y valor nulo. En las longitudes inválidas se comprueba el error 400 y que el modelo no recibió la orden de crear.
Pruebas de Jest que lo cubren	· «guarda los 13 dígitos sin separadores» · «lo deja vacío si no lo traen» (tres variantes) · «rechaza ... porque no tiene 13 dígitos» (tres variantes)
Caso del plan de pruebas	CP-PAC-004
Cómo ejecutarlo solo	<code>./caso.sh PAC004</code> Equivale a: <code>cd backend && npx jest paciente.service -t "DPI" --verbose</code>

#PAC005 Un paciente cambia de número de teléfono, y el sistema deja de dar por sabido si el nuevo número tiene WhatsApp.	
ID del caso de prueba	#PAC005
Escenario de prueba	Un paciente cambia de número de teléfono, y el sistema deja de dar por sabido si el nuevo número tiene WhatsApp.
Pasos de la prueba	1. Un paciente registrado avisa al laboratorio de que cambió de número. 2. El personal de citas busca su ficha y hace clic en «Editar». 3. Cambia el teléfono por el nuevo y guarda. 4. Observa cómo queda el indicador de WhatsApp de ese paciente. 5. En otra ficha distinta, corrige solo una falta de ortografía en el nombre y guarda.
Requisitos previos	Un paciente registrado del que ya se sabía si su número anterior tenía WhatsApp. El sistema usa ese dato para decidir si manda el recordatorio por mensaje o hay que llamarlo.
Navegador / Dispositivo	MacBook Air · Apple M4 · 16 GB RAM · macOS 26.6.2 · Jest 29.7.0 · entorno «node» · Node.js v24.14.0 · sin navegador
Datos de prueba	Cambio de teléfono al «23232323». En la segunda ficha, cambio de nombre a «Ana María López».
Resultados esperados	Al cambiar el número, el indicador de WhatsApp vuelve a «se desconoce»: el dato pertenecía al número anterior y ya no sirve. Al corregir solo el nombre, el indicador se queda exactamente como estaba.
Resultados reales	Como se esperaba en los dos casos.
Estado de la prueba	APROBADO
Tipo de prueba	Caja blanca — unitaria (regla de consistencia) La prueba mira el objeto exacto con el que se invocó al modelo y distingue entre «se guardó desconocido» y «no se envió el campo»: una distinción que solo existe por dentro.
Cómo se automatiza con Jest	Se llama a <code>paciente.service.actualizar</code> y se recoge la llamada que recibió el modelo. En el primer caso se comprueba que incluye <code>tieneWhatsapp</code> en nulo; en el segundo, que la propiedad ni siquiera aparece.
Pruebas de Jest que lo cubren	· «olvida lo que sabía del WhatsApp al cambiar el número» · «no toca el dato de WhatsApp si solo cambia el nombre»
Caso del plan de pruebas	CP-PAC-006

Cómo ejecutarlo solo	./caso.sh PAC005 Equivale a: <code>cd backend && npx jest paciente.service -t "WhatsApp" --verbose</code>
----------------------	--

#PAC006 El formulario de paciente avisa de los datos mal escritos en la misma pantalla, sin esperar al servidor.

ID del caso de prueba	#PAC006
Escenario de prueba	El formulario de paciente avisa de los datos mal escritos en la misma pantalla, sin esperar al servidor.
Pasos de la prueba	1. El personal de citas abre «Nuevo paciente». 2. Escribe el nombre del paciente. 3. Escribe un teléfono de 7 dígitos y hace clic en «Guardar». 4. Lee el mensaje que aparece bajo el campo del teléfono. 5. Corrige el teléfono por uno válido de 8 dígitos y vuelve a guardar. 6. Por último, intenta guardar dejando el nombre vacío y solo con el teléfono.
Requisitos previos	Sesión iniciada con permiso para registrar pacientes.
Navegador / Dispositivo	MacBook Air · Apple M4 · 16 GB RAM · macOS 26.6.2 · Jest 29.7.0 · entorno «jsdom» (navegador simulado) · Testing Library 16.3 + user-event 14.5 · Node.js v24.14.0
Datos de prueba	Teléfono inválido «2323232». Teléfonos válidos «23232323» y «55555555». Nombre vacío en la última variante.
Resultados esperados	Con el teléfono corto aparece el mensaje «El teléfono debe ser un número de Guatemala de 8 dígitos.» y el formulario no llega a enviar nada al servidor. Con los teléfonos válidos sí se envía, con el número tal como se escribió. Sin nombre aparece «El nombre completo es obligatorio.».
Resultados reales	Como se esperaba en las cuatro variantes.
Estado de la prueba	APROBADO
Tipo de prueba	Caja negra — funcional (validación de entrada) Se rellena el formulario por las etiquetas visibles y se leen los mensajes que vería el personal del mostrador.
Cómo se automatiza con Jest	El diálogo se dibuja en el navegador simulado y user-event escribe y pulsa como una persona. El servicio de pacientes está sustituido por un doble, lo que permite comprobar si la pantalla llegó a llamarlo o si detuvo el envío por su cuenta.
Pruebas de Jest que lo cubren	· «<i>valida el teléfono en el cliente antes de llamar al backend</i>» · «<i>acepta el teléfono válido</i>» (dos variantes) · «<i>exige el nombre completo</i>»
Caso del plan de pruebas	CP-PAC-002 y CP-PAC-007
Cómo ejecutarlo solo	./caso.sh PAC006 Equivale a: <code>cd frontend && BABEL_ENV=test npx jest PacienteFormDialog -t "valida el teléfono en el cliente acepta el teléfono válido exige el nombre completo" --verbose</code>

#PAC007 Editar la ficha de un paciente existente y ver en pantalla el aviso de

teléfono repetido que manda el servidor.	
ID del caso de prueba	#PAC007
Escenario de prueba	Editar la ficha de un paciente existente y ver en pantalla el aviso de teléfono repetido que manda el servidor.
Pasos de la prueba	1. El personal de citas busca a un paciente en el listado. 2. Abre su menú de acciones y elige «Editar». 3. Comprueba que el formulario ya viene con el nombre, el teléfono, el DPI y las notas del paciente. 4. Después, en un alta nueva, registra a un paciente con un teléfono que ya está en uso. 5. Hace clic en «Guardar» y lee la notificación que aparece.
Requisitos previos	Un paciente ya registrado con nombre, teléfono, DPI y notas; y otro paciente que comparte teléfono con el que se va a dar de alta.
Navegador / Dispositivo	MacBook Air · Apple M4 · 16 GB RAM · macOS 26.6.2 · Jest 29.7.0 · entorno «jsdom» (navegador simulado) · Testing Library 16.3 + user-event 14.5 · Node.js v24.14.0
Datos de prueba	Paciente a editar: «Carlos Ruiz», teléfono «44444444», DPI «111111111111», notas «Alergias». Aviso del servidor: «Ya existen 1 paciente(s) registrados con ese teléfono: Carlos Pérez.».
Resultados esperados	Al editar, los campos llegan ya rellenos con los datos del paciente y el diálogo se titula «Editar paciente», no «Nuevo paciente». En el alta, aparece una notificación de advertencia que nombra al paciente que ya usaba ese teléfono.
Resultados reales	Como se esperaba en los dos escenarios.
Estado de la prueba	APROBADO
Tipo de prueba	Caja negra — funcional (integración de pantalla y servicio) Se verifica lo que el usuario ve: los campos precargados y el texto del aviso.
Cómo se automatiza con Jest	Al diálogo se le pasa el paciente a editar y se leen los valores de cada campo por su etiqueta. Para el aviso, el doble del servicio devuelve la lista de advertencias y se comprueba que la notificación mostrada contiene el nombre esperado.
Pruebas de Jest que lo cubren	· «<i>precarga los datos al editar</i>» · «<i>muestra el aviso de teléfono repetido que devuelve el backend</i>»
Caso del plan de pruebas	CP-PAC-003 y CP-PAC-008
Cómo ejecutarlo solo	./caso.sh PAC007 Equivale a: cd frontend && BABEL_ENV=test npx jest PacienteFormDialog -t "precarga los datos al editar muestra el aviso de teléfono repetido" --verbose

5. Módulo 3 — Citas y Recepción

Cubre la recepción en el mostrador, que es la operación más delicada del sistema: en un solo paso se registra al paciente, su orden del IGSS y la cita. Se prueban las validaciones que protegen la vigencia de la orden y el cupo del día, y el deshacer cuando algo falla a mitad.

Archivos de prueba: `backend/src/tests/recepcion.service.test.js`

#CIT001 Recibir en el mostrador a un paciente nuevo que llega con su orden del IGSS y dejarle la cita agendada.	
ID del caso de prueba	#CIT001
Escenario de prueba	Recibir en el mostrador a un paciente nuevo que llega con su orden del IGSS y dejarle la cita agendada.
Pasos de la prueba	1. Un paciente que nunca ha venido llega al mostrador con su orden del IGSS. 2. El personal de citas abre la pantalla de recepción. 3. Escribe los datos del paciente: nombre, teléfono y DPI. 4. Registra la orden, con su fecha de recepción y los exámenes que pide. 5. Elige la fecha y la hora de la cita. 6. Hace clic en «Guardar».
Requisitos previos	Sesión iniciada con permiso para recibir citas. El laboratorio tiene configurada la vigencia de las órdenes en 3 meses.
Navegador / Dispositivo	MacBook Air · Apple M4 · 16 GB RAM · macOS 26.6.2 · Jest 29.7.0 · entorno «node» · Node.js v24.14.0 · sin navegador
Datos de prueba	Paciente nuevo con teléfono válido, un examen del catálogo y una fecha de recepción dentro del periodo permitido.
Resultados esperados	En una sola operación quedan registrados el paciente, su orden y su cita, enlazados entre sí. La fecha de vencimiento de la orden se calcula sumando la vigencia a la fecha en que se recibió la orden, no a la fecha de hoy.
Resultados reales	Como se esperaba.
Estado de la prueba	APROBADO
Tipo de prueba	Caja blanca — de integración entre servicios Se comprueba la coordinación interna entre varios modelos y el cálculo de una fecha derivada.
Cómo se automatiza con Jest	Los modelos de paciente, orden, cita y examen están sustituidos por dobles de prueba, y la configuración del laboratorio se simula con la vigencia en 3 meses. Se llama al servicio de recepción y se comprueba qué recibió cada modelo y qué vencimiento se calculó.
Pruebas de Jest que lo cubren	· «registra paciente, orden y cita de una sola vez» · «calcula el vencimiento de la orden desde la fecha de recepción»
Caso del plan de pruebas	CP-CIT-001
Cómo ejecutarlo solo	./caso.sh CIT001 Equivale a: <code>cd backend && npx jest recepcion.service -t "registra paciente, orden y cita de una sola vez calcula el vencimiento de la orden" --verbose</code>

#CIT002 Intentar agendar una cita para después de que

venza la orden del IGSS, y recibir fechas alternativas válidas.	
ID del caso de prueba	#CIT002
Escenario de prueba	Intentar agendar una cita para después de que venza la orden del IGSS, y recibir fechas alternativas válidas.
Pasos de la prueba	1. Llega al mostrador un paciente con una orden que se recibió hace tiempo y está por vencer. 2. El personal de citas elige para la cita una fecha posterior al vencimiento de la orden. 3. Hace clic en «Guardar». 4. Lee el aviso y las fechas que el sistema propone en su lugar. 5. Repite la operación eligiendo exactamente el último día de vigencia de la orden.
Requisitos previos	Una orden del IGSS con fecha de vencimiento conocida. El IGSS no acepta exámenes hechos con la orden ya vencida.
Navegador / Dispositivo	MacBook Air · Apple M4 · 16 GB RAM · macOS 26.6.2 · Jest 29.7.0 · entorno «node» · Node.js v24.14.0 · sin navegador
Datos de prueba	Una fecha de cita posterior al vencimiento; y otra situada exactamente en el último día de vigencia.
Resultados esperados	La cita posterior al vencimiento se rechaza y el sistema ofrece fechas alternativas, todas anteriores al vencimiento. La cita del último día sí se acepta, pero acompañada de un aviso para que el personal se lo advierta al paciente. Unos días antes del vencimiento no se avisa nada.
Resultados reales	Como se esperaba en los tres escenarios.
Estado de la prueba	APROBADO
Tipo de prueba	Caja gris — valores límite sobre una regla de negocio El borde probado —el día exacto del vencimiento— sale de la regla del IGSS, pero la comprobación de las fechas sugeridas exige conocer el algoritmo que las genera.
Cómo se automatiza con Jest	Se llama al servicio de recepción con cada fecha y se comprueba el error devuelto y la lista de fechas sugeridas que lo acompaña, verificando que ninguna supere el vencimiento.
Pruebas de Jest que lo cubren	· «rechaza una cita posterior al vencimiento y sugiere fechas» · «avisa cuando la cita cae el último día de vigencia» · «no avisa nada raro por agendar unos días antes del vencimiento»
Caso del plan de pruebas	CP-CIT-004
Cómo ejecutarlo solo	./caso.sh CIT002 Equivale a: cd backend && npx jest recepcion.service -t "rechaza una cita posterior al vencimiento último día de vigencia no avisa nada raro" --verbose

#CIT003 Intentar recibir a un paciente para un día que ya alcanzó su cupo máximo de atenciones.	
ID del caso de prueba	#CIT003
Escenario de prueba	Intentar recibir a un paciente para un día que ya alcanzó su cupo máximo de atenciones.
Pasos de la prueba	1. El personal de citas recibe a un paciente y elige una fecha que ya tiene el cupo completo. 2. Hace clic en «Guardar».

Requisitos previos	3. Lee el aviso del sistema. 4. Va al módulo «Pacientes» y comprueba si quedó alguna ficha registrada a medias.
	Un día de la agenda cuya ocupación ya iguala el límite diario de pacientes que tiene configurado el laboratorio.
Navegador / Dispositivo	MacBook Air · Apple M4 · 16 GB RAM · macOS 26.6.2 · Jest 29.7.0 · entorno «node» · Node.js v24.14.0 · sin navegador
Datos de prueba	Un día lleno. También se prueba con una fecha de recepción futura y sin indicar ningún examen.
Resultados esperados	La operación se rechaza con un aviso y no queda registrado ni el paciente, ni la orden, ni la cita. El sistema comprueba todas las condiciones antes de empezar a escribir, así que no deja fichas huérfanas.
Resultados reales	Como se esperaba. No se escribió nada.
Estado de la prueba	APROBADO
Tipo de prueba	Caja blanca — unitaria (orden de validaciones) Lo que se afirma no es el mensaje de error, sino que no hubo escrituras: eso obliga a observar las llamadas internas a los modelos.
Cómo se automatiza con Jest	Se prepara la ocupación del día en el límite y se llama al servicio. Después se comprueba que ninguno de los dobles de los modelos recibió una orden de crear.
Pruebas de Jest que lo cubren	· «rechaza el día lleno sin haber creado nada» · «rechaza una fecha de recepción futura» · «rechaza si no se indica ningún examen»
Caso del plan de pruebas	CP-CIT-003
Cómo ejecutarlo solo	./caso.sh CIT003 Equivale a: cd backend && npx jest recepcion.service -t "rechaza el día lleno rechaza una fecha de recepción futura rechaza si no se indica ningún examen" --verbose

#CIT004 La recepción falla en el último paso y el sistema deshace el paciente y la orden que acababa de crear.

ID del caso de prueba	#CIT004
Escenario de prueba	La recepción falla en el último paso y el sistema deshace el paciente y la orden que acababa de crear.
Pasos de la prueba	1. El personal de citas recibe a un paciente nuevo con su orden y pulsa «Guardar». 2. En el último paso, al crear la cita, ocurre un fallo: se pierde la conexión con la base de datos. 3. El personal va al módulo «Pacientes» y busca al paciente que estaba registrando. 4. Comprueba también si quedó suelta la orden del IGSS. 5. Repite la misma situación, pero con un paciente que ya estaba registrado de antes.
Requisitos previos	Sesión iniciada con permiso para recibir citas. Este escenario no se puede provocar a voluntad desde la pantalla: depende de un fallo real del sistema.
Navegador / Dispositivo	MacBook Air · Apple M4 · 16 GB RAM · macOS 26.6.2 · Jest 29.7.0 · entorno «node» · Node.js v24.14.0 · sin navegador

Datos de prueba	Escenario 1: paciente nuevo y fallo al crear la cita. Escenario 2: paciente ya registrado y el mismo fallo.
Resultados esperados	En el primer caso no queda rastro: ni el paciente ni la orden, porque se crearon en esta operación y la operación no llegó a completarse. En el segundo, el paciente que ya existía se conserva intacto, porque no se creó aquí y borrarlo destruiría su historial.
Resultados reales	Como se esperaba en los dos escenarios.
Estado de la prueba	APROBADO
Tipo de prueba	Caja blanca — unitaria (compensación ante error) Es una ruta de excepción que no se puede provocar desde la interfaz; se fuerza el fallo desde dentro y se observan las llamadas de borrado.
Cómo se automatiza con Jest	Se configura el doble del modelo de cita para que lance un error al crear. Después se comprueba qué borrados se llamaron: en el primer caso los del paciente y la orden recién creados; en el segundo, ninguno sobre el paciente preexistente.
Pruebas de Jest que lo cubren	· «borra la orden y el paciente recién creados» · «no borra a un paciente que ya existía»
Caso del plan de pruebas	CP-CIT-012
Cómo ejecutarlo solo	./caso.sh CIT004 Equivale a: cd backend && npx jest recepcion.service -t "Deshacer si la cita falla al final" --verbose

#CIT005 Anotar en el mostrador si el número del paciente tiene WhatsApp, incluido el caso de que no se le haya preguntado.	
ID del caso de prueba	#CIT005
Escenario de prueba	Anotar en el mostrador si el número del paciente tiene WhatsApp, incluido el caso de que no se le haya preguntado.
Pasos de la prueba	1. El personal de citas recibe a un paciente y le pregunta si su número tiene WhatsApp. 2. El paciente responde que sí; el personal marca la casilla y guarda. 3. Con el siguiente paciente, que responde que no, el personal marca esa opción y guarda. 4. Con un tercero hay prisa y no se le llega a preguntar: la casilla se queda sin marcar. 5. Por último, un paciente ya registrado avisa de que ahora sí tiene WhatsApp, y el personal lo actualiza.
Requisitos previos	Sesión iniciada con permiso para recibir citas. El sistema manda los recordatorios por WhatsApp; si no sabe si el número lo tiene, hay que llamar al paciente por teléfono.
Navegador / Dispositivo	MacBook Air · Apple M4 · 16 GB RAM · macOS 26.6.2 · Jest 29.7.0 · entorno «node» · Node.js v24.14.0 · sin navegador
Datos de prueba	Los tres estados posibles del campo: sí, no y sin preguntar.
Resultados esperados	Los tres estados se guardan tal cual, incluido el de «no se preguntó», que es el que hace que el sistema mande a llamar al paciente en vez de escribirle. Si el paciente ya existía y su respuesta cambia, el dato se actualiza; si coincide con lo que ya había, no se vuelve a escribir.
Resultados reales	Como se esperaba en las cinco variantes.
Estado de la prueba	APROBADO
Tipo de prueba	Caja negra — partición de equivalencia (tres estados)

	Las clases probadas salen directamente de la especificación del campo, que admite tres valores y no dos.
Cómo se automatiza con Jest	Se llama al servicio de recepción con cada uno de los tres valores y se comprueba lo que recibió el modelo del paciente. En las dos últimas variantes se verifica si la actualización llegó a ocurrir o si el sistema la evitó por innecesaria.
Pruebas de Jest que lo cubren	· «guarda que el número sí tiene WhatsApp» · «guarda que no lo tiene, que es lo que obliga a llamar» · «lo deja en desconocido si no se preguntó» · «actualiza el dato de un paciente ya registrado» · «no reescribe el dato si ya coincidía»
Caso del plan de pruebas	CP-CIT-002 y CP-REC-001
Cómo ejecutarlo solo	./caso.sh CIT005 Equivale a: <pre>cd backend && npx jest recepcion.service -t "Confirmación de WhatsApp en el mostrador" --verbose</pre>

6. Cobertura de código de los módulos evaluados

La cobertura es la medida propia de la caja blanca: dice qué porcentaje de las líneas y de las ramas de decisión del código llegó a ejecutarse durante las pruebas. Se obtiene con `npm run test:coverage` y estos son los archivos que sostienen los casos documentados arriba.

ARCHIVO EVALUADO	MÓDULO	% LÍNEAS	% RAMAS	% FUNCIONES
<i>auth.service.js</i>	Autenticación	97.43	87.50	100
<i>jwt.js</i>	Autenticación	100	100	100
<i>password.js</i>	Autenticación	100	100	100
<i>LoginPage.jsx</i>	Autenticación (pantalla)	100	100	100
<i>paciente.service.js</i>	Pacientes	90.47	66.07	85.71
<i>telefono.js</i>	Pacientes	100	91.66	100
<i>dpi.js</i>	Pacientes	100	92.30	100
<i>PacienteFormDialog.jsx</i>	Pacientes (pantalla)	83.33	61.29	87.50
<i>recepcion.service.js</i>	Citas y Recepción	96.15	92.10	100
<i>cita.service.js</i>	Citas y Recepción	94.59	90.29	87.50
<i>disponibilidad.service.js</i>	Citas y Recepción	100	94.73	100
<i>RecepcionCitaDialog.jsx</i>	Citas (pantalla)	86.04	80.70	71.18

La cobertura global del backend es del 70.95 % de líneas. No se persigue el 100 %: un número redondo se consigue fácilmente escribiendo pruebas que no comprueban nada. Lo que se buscó fue cubrir los caminos donde un fallo tiene consecuencias reales para el laboratorio: la verificación de credenciales, la validación de los datos del paciente y las reglas que protegen la vigencia de la orden del IGSS.

7. Resumen de la ejecución completa

Las fichas anteriores detallan los casos de tres módulos, pero la ejecución que se ve en el video cubre el proyecto entero. Este es el recuento por proyecto:

PROYECTO	QUÉ CUBRE	SUITES	PRUEBAS	RESULTADO
backend	Servicios, rutas, validadores y utilidades de la API	17	370	Aprobado
frontend	Pantallas y componentes de React	11	96	Aprobado
whatsapp-service	Envío de recordatorios y sesión de WhatsApp	5	64	Aprobado
TOTAL		33	530	530 / 530

8. Evidencia en video

Todos los comandos se ejecutan desde la raíz del proyecto, sin cambiar de carpeta: tanto el lanzador de casos como npm saben por sí solos en qué proyecto vive cada prueba.

1. Listado de los casos documentados.
`./caso.sh`
2. Un caso de caja blanca: el token que emite el login.
`./caso.sh AUT001`
3. Un caso de caja gris: el error del login no delata qué usuarios existen.
`./caso.sh AUT004`
4. Un caso de caja negra: los valores límite del teléfono.
`./caso.sh PAC002`
5. Un caso de caja negra sobre la interfaz, en el navegador simulado.
`./caso.sh AUT005`
6. La suite completa del backend: 17 suites y 370 pruebas.
`npm test --prefix backend`
7. La suite completa del frontend: 11 suites y 96 pruebas.
`npm test --prefix frontend`
8. Una prueba en rojo: se afloja a propósito la regla del teléfono en backend/src/utils/telefono.js (de {8} a {7,9} dígitos), se vuelve a ejecutar y fallan cinco pruebas en dos suites. Después se restaura el archivo y todo vuelve a verde.
`npm test --prefix backend -- telefono paciente.service`
9. El informe de cobertura, que es la medida de la caja blanca.
`npm run test:coverage --prefix backend`

El paso 8 es el que demuestra que las pruebas sirven de algo: una prueba que nunca ha fallado no ha demostrado todavía que esté vigilando nada. Conviene enseñar la ficha del documento antes de lanzar cada caso, leyendo el «Resultado esperado» y comparándolo después con lo que imprime la terminal: ese es exactamente el contrato que describe este formato.